Quantitative Research

Part 5: Classification

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

The lecture at a glance

	Part	Content
	1 – Descriptive Statistics	Basic concepts, levels of measurement, frequencies, measures of central tendency and dispersion
	2 – Graphical Representations	Empirical CDF, histogram, boxplot
	3 – Statistical Inference	Confidence intervals, hypothesis tests, p-values, one-sided tests, types of error
	4 – Regression	Correlation, simple and multiple linear regression, inference for regression, diagnostics
▶	**5 – Classification**	**Logistic regression, classification rules, confusion matrix, ROC/AUC, Bayes classifiers**

All analyses are carried out in **Python** (NumPy, pandas, SciPy, statsmodels, matplotlib); every deck comes with a Python script that reproduces its numbers.

Contents – Part 5

1. Classification Problems
2. Logistic Regression
3. Multiple Logistic Regression
4. From Probabilities to Decisions
5. Evaluating a Classifier
6. Summary

The common thread of this part

One question runs through every section

The variable Y is no longer metric but **categorical** (yes/no). How do we estimate the **probability** of "yes" – and how do we turn it into a good **decision**?

Three steps:

- **Model** – logistic regression: a linear model for the log-odds (Part 4 on a new scale).
- **Decide** – a threshold turns a probability into a class.
- **Evaluate** – confusion matrix, sensitivity, specificity, ROC curve.

One running story:

Credit card default

A bank observes $n = 10\,000$ customers: did they **default** on their credit card debt? Explanatory variables: the card **balance**, the **income** and whether the customer is a **student**.

Learning objectives – Part 5

After this part you will be able to ...

- explain why **linear regression fails** for a 0/1 variable,
- convert between **probability, odds and log-odds**,
- interpret the coefficients of a **logistic regression** (sign, odds ratio, probability),
- carry out z-**tests** and compute **confidence intervals** for coefficients and odds ratios,
- turn probabilities into decisions with a **threshold**,
- compute **accuracy, sensitivity, specificity and precision** from a confusion matrix,
- read a **ROC curve** and interpret the **AUC**,
- evaluate a classifier honestly on **test data**,
- carry out every step in Python.

Key takeaway

Common thread: classification = estimating a probability *plus* a decision rule – and both have to be evaluated honestly.

→ *Appendix: Bayes' theorem for classification, naive Bayes, LDA*

List of symbols (1/2)

Symbol	Meaning	Symbol	Meaning
$Y \in \{0, 1\}$	Binary response ($1 =$ "yes")	$X_1, \ldots, X_p$	Explanatory variables
$p(x)$	$P(Y = 1 \mid X = x)$	$\hat{p}(x)$	Estimated probability
$\frac{p}{1-p}$	Odds	$\log \frac{p}{1-p}$	Log-odds (logit)
$\eta = \beta_0 + \beta_1 x$	Linear predictor (log-odds)	e^{β_j}	Odds ratio
$L(\beta)$	Likelihood	$\hat{\beta}_j$	Maximum likelihood estimate
Z	z-statistic $\hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j)$	$z_{1-\alpha/2}$	Quantile of $N(0, 1)$

List of symbols (2/2)

Symbol	Meaning	Symbol	Meaning
t	Threshold of the classification rule	$\hat{y}$	Predicted class
TP, TN	True positives / negatives	FP, FN	False positives / negatives
Sens.	Sensitivity $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$	Spec.	Specificity $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$
ROC	Receiver operating characteristic	AUC	Area under the ROC curve
π_k	Prior probability of class k	$f_k(x)$	Density of X in class k

As in Parts 3 and 4: H_0, H_1, α, p-value, confidence level $1-\alpha$, standard error SE.

Overview

1 Classification Problems
- What is classification?
- Why not linear regression?

2 Logistic Regression

3 Multiple Logistic Regression

4 From Probabilities to Decisions

5 Evaluating a Classifier

6 Summary

From regression to classification

Regression (Part 4)

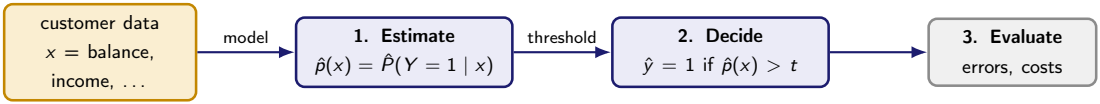
The variable to be explained Y is **metric**: sales quantity, shoe size, card balance.

Classification (Part 5)

Y is **categorical** – in this part *binary*: yes/no, coded $Y = 1$ ("yes") and $Y = 0$ ("no").

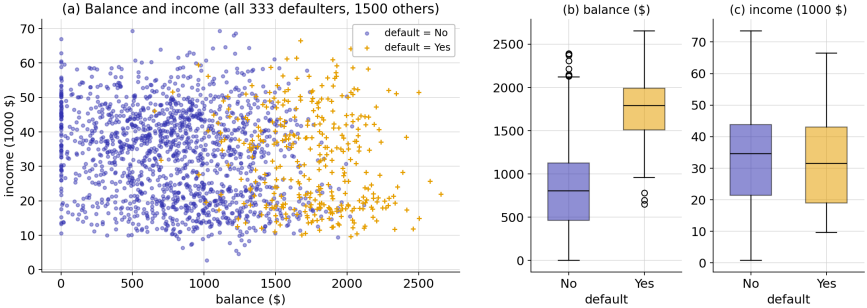
Question	Y	**Explanatory variables**
Will the customer default?	default yes/no	balance, income, student
Is this e-mail spam?	spam yes/no	words, links, sender
Will the customer churn?	churn yes/no	usage, contract, complaints
Is the patient ill?	diagnosis yes/no	test results, age

Two tasks in every classification problem



- The **probability** is a statistical question – answered by a model such as logistic regression (Sections 2–3).
- The **decision** is also a business question – which error is more expensive? (Sections 4–5)

Our story: the Default data



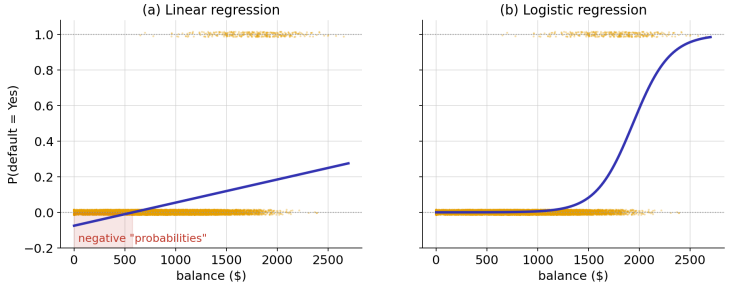
- $n = 10\,000$ customers, of which only 333 (3.33 %) defaulted.
- Defaulters have clearly **higher balances**; income hardly differs between the groups.

Source: Default data from James, Witten, Hastie, Tibshirani: An Introduction to Statistical Learning, www.statlearning.com

Why not simply use linear regression?

Tempting idea

Code $Y = 1$ for default and $Y = 0$ otherwise and run the regression of Part 4: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1$ balance.



In words: The straight line predicts *negative* default "probabilities" for 3123 customers (all balances below 579 $); the S-curve stays between 0 and 1.

Three problems of linear regression with a 0/1 variable

Problems

1. **Predictions outside** $[0, 1]$ – they cannot be read as probabilities.
2. **Heteroscedasticity by construction** – $\text{Var}(Y \mid x) = p(x)\,(1 - p(x))$ depends on x, so the standard errors and tests of Part 4 are not valid.
3. **No natural extension to more than two categories** – codes $1, 2, 3$ would impose an order and equal distances that the categories do not have.

Key takeaway

We need a model whose output is always a probability in $(0, 1)$ – the **logistic regression**.

Exercise – why not linear regression?

Problem statement

A colleague wants to predict the emergency diagnosis of a patient. She codes the three *unordered* categories as $Y = 1$ (stroke), $Y = 2$ (overdose), $Y = 3$ (seizure) and fits a linear regression.

1. Which implicit assumptions does the coding $1, 2, 3$ make?
2. With only two categories coded $0/1$: name two problems of the fitted values as probabilities.
3. Would relabelling the categories change the fit? What does this imply?

Solution – why not linear regression?

1. **Order and equal spacing:** the coding claims stroke $<$ overdose $<$ seizure and equal distances between them. A fitted value of 1.5 would mean "halfway between stroke and overdose" – medically meaningless.
2. **Two problems:** fitted values can lie outside $[0, 1]$ (e.g. a negative probability), and the error variance $p(1 - p)$ is not constant.
3. **Yes:** swapping the codes of stroke and seizure gives a different response and hence a different fit – although the data are the same. The result depends on an arbitrary choice, so linear regression is *not appropriate* here.

Common mistake

Hoping that more data fix the problem. The flaw lies in the model specification, not in sampling noise.

Overview

1 Classification Problems

2 Logistic Regression
- Probability, odds and log-odds
- The model
- Estimation
- Interpretation
- Inference for the coefficients

3 Multiple Logistic Regression

4 From Probabilities to Decisions

5 Evaluating a Classifier

6 Summary

Odds – another way to express a probability

Definition

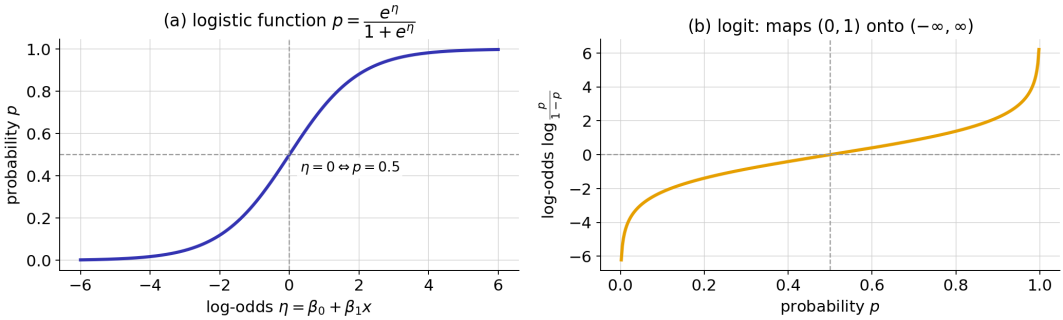
For an event with probability p:

$$\text{odds} = \frac{p}{1-p} \in (0, \infty), \qquad \text{log-odds (logit)} = \log \frac{p}{1-p} \in (-\infty, \infty).$$

Probability p	0.1	0.2	0.5	0.8	0.9
Odds $p/(1-p)$	0.111	0.25	1	4	9
Log-odds	-2.197	-1.386	0	1.386	2.197

In words: Odds of 4 mean "4 to 1": the event occurs four times as often as it does not ($p = 0.8$). Back from odds to probability: $p = \text{odds}/(1 + \text{odds})$.

Why log-odds?



- The log-odds can take **any real value** – just like the right-hand side $\beta_0 + \beta_1 x$ of a regression.
- The logistic function (a) is the inverse of the logit (b): it maps every log-odds value back into $(0, 1)$.

The logistic regression model

Model: linear on the log-odds scale

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + \beta_1 x \qquad \Longleftrightarrow \qquad p(x) = P(Y=1 \mid x) = \frac{e^{\beta_0+\beta_1 x}}{1+e^{\beta_0+\beta_1 x}}$$

- Both forms describe the *same* model: the left one shows the linearity, the right one guarantees $0 < p(x) < 1$.
- We write $\eta = \beta_0 + \beta_1 x$ for the **linear predictor** (the log-odds).
- Several explanatory variables: $\eta = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$ (Section 3).

Key takeaway

Logistic regression = the regression model of Part 4, applied to the **log-odds** instead of Y itself.

Reading the model symbol by symbol

Symbol	Meaning
Y	Binary response, 0 or 1 (default no/yes)
$p(x)$	$P(Y = 1 \mid x)$, the probability of default given x
$p(x)/(1 - p(x))$	The *odds* of default
$\eta = \log[p/(1 - p)]$	The *log-odds*, linear in x
β_1	Change of the *log-odds* per unit of x
e^{β_1}	Factor by which the *odds* change per unit of x

In words: Every probability statement goes through the pipeline

$$x \xrightarrow{\text{linear}} \eta \xrightarrow{\exp} \text{odds} \xrightarrow{\text{odds}/(1+\text{odds})} p.$$

Estimation: maximum likelihood

Idea

Least squares makes no sense for 0/1 data. Instead we choose the coefficients under which the **observed outcomes are most probable**.

Likelihood

$$L(\beta_0, \beta_1) = \prod_{i:\, y_i = 1} p(x_i) \prod_{i:\, y_i = 0} \big(1 - p(x_i)\big)$$

Each defaulter contributes $p(x_i)$, each non-defaulter $1 - p(x_i)$.

- In practice we maximise the **log-likelihood** $\log L = \sum_i \big[y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i))\big]$.
- There is no closed formula – Python solves it numerically (`smf.logit(...).fit()`).

Exercise – which parameters are more plausible?

Problem statement

Four customers with balances 1000, 1500, 2000, 2500 \$; the last two defaulted ($y = 0, 0, 1, 1$). Compare two candidate parameter sets:

$$\text{(A)}\ \ \beta_0 = -10.65,\ \beta_1 = 0.0055 \qquad\qquad \text{(B)}\ \ \beta_0 = -5,\ \beta_1 = 0.002$$

1. Compute $p(x_i)$ for all four customers under (A) and (B).
2. Compute the likelihood L for (A) and (B).
3. Which parameter set would maximum likelihood prefer?

Solution – which parameters are more plausible?

Example: $\eta(2000) = -10.65 + 0.0055 \cdot 2000 = 0.35$, $p = e^{0.35}/(1 + e^{0.35}) = 0.5866$.

		(A)		**(B)**	
balance	y_i	$p(x_i)$	contribution	$p(x_i)$	contribution
1000	0	0.0058	0.9942	0.0474	0.9526
1500	0	0.0832	0.9168	0.1192	0.8808
2000	1	0.5866	0.5866	0.2689	0.2689
2500	1	0.9569	0.9569	0.5000	0.5000
L (product)		**0.5117**		0.1128	

Decision

Under (A) the observed pattern is 4.5 times as probable as under (B) $\Rightarrow$ maximum likelihood prefers **(A)**: high probabilities for the actual defaulters, low ones for the others.

Logistic regression with Python

Fitting the model

```
import pandas as pd
import statsmodels.formula.api as smf
df = pd.read_csv("../data/Default.csv")
df["y"] = (df.default == "Yes").astype(int)      # 1 = default, 0 = no
g1 = smf.logit("y ~ balance", data=df).fit(disp=0)
print(g1.summary2().tables[1])
```

	Coef.	Std.Err.	z	P>|z|	[0.025	0.975]
Intercept	-10.651331	0.361169	-29.491287	3.723665e-191	-11.359208	-9.943453
balance	0.005499	0.000220	24.952404	2.010855e-137	0.005067	0.005931

- `smf.logit` fits a logistic regression by maximum likelihood (y coded 0/1).
- Estimated model: $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$.
- The output looks like `smf.ols` – with z- instead of t-values (see "Inference").

Interpreting the coefficients – three scales

Three ways to read $\hat{\beta}_1$

Sign: $\hat{\beta}_1 > 0 \Rightarrow$ a larger x raises the probability of $Y = 1$; $\hat{\beta}_1 < 0$ lowers it.

Log-odds: one unit more of x changes the log-odds by $\hat{\beta}_1$ – constant, but hard to imagine.

Odds: one unit more of x **multiplies the odds** by $e^{\hat{\beta}_1}$ **(odds ratio)** – constant and easy to communicate.

Not on the probability scale

The change in the *probability* is not constant: it is largest in the steep middle of the S-curve and small at the flat ends.

In words: Why the odds ratio? $\frac{\text{odds}(x+1)}{\text{odds}(x)} = \frac{e^{\beta_0+\beta_1(x+1)}}{e^{\beta_0+\beta_1 x}} = e^{\beta_1}$ – independent of x.

Worked example: default and balance

Model: $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$.

Three scales for 1000 $ more balance

- **Log-odds:** $+0.0055 \cdot 1000 = +5.5$.
- **Odds:** multiplied by $e^{5.5} \approx 245$.
- **Probability:** depends on the starting point:

balance ($)	1000	1500	2000	2500
$\hat{p}(x)$	0.006	0.083	0.586	0.957

Common mistake

"The odds multiply by 245" does *not* mean the probability is 245 times as large – probabilities can never exceed 1.

Exercise – computing $\hat{p}(x)$ from the coefficients

Problem statement

The logistic regression of default on balance gives $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$. For a customer with a balance of 1500 \$:

1. Compute the log-odds $\hat{\eta}$.
2. Compute the probability of default $\hat{p}$.
3. Compute the odds and relate them to part (1).
4. At which balance does the model give $\hat{p} = 0.5$?

Solution – computing $\hat{p}(x)$ from the coefficients

(1)–(3) Log-odds, probability, odds

$$\hat{\eta} = -10.65 + 0.0055 \cdot 1500 = -2.40, \qquad \hat{p} = \frac{e^{-2.40}}{1 + e^{-2.40}} = \frac{0.0907}{1.0907} = \mathbf{0.083},$$

$$\text{odds} = \frac{0.083}{0.917} = 0.091 = e^{-2.40} \ \checkmark \quad \text{(about one default per eleven non-defaults)}$$

(4) Balance with $\hat{p} = 0.5$

$\hat{p} = 0.5 \Leftrightarrow \text{odds} = 1 \Leftrightarrow \hat{\eta} = 0$:
$0 = -10.65 + 0.0055 \cdot \text{balance} \Rightarrow \text{balance} = 10.65/0.0055 \approx \mathbf{1936}\,\$$.

Common mistake

Reading $\hat{\eta} = -2.40$ as a probability – log-odds must first be passed through the logistic function.

Inference – the same recipe once more

z-test for a coefficient (Wald test)

$$H_0 : \beta_j = 0 \quad \text{vs.} \quad H_1 : \beta_j \neq 0, \qquad Z = \frac{\hat{\beta}_j}{\mathrm{SE}(\hat{\beta}_j)} \;\approx\; N(0,1) \;\text{ under } H_0$$

Reject if $|Z| > z_{1-\alpha/2}$ (1.96 for $\alpha = 0.05$); $p = 2\,\Phi(-|Z|)$.

Confidence intervals

$$\beta_j : \quad \hat{\beta}_j \pm z_{1-\alpha/2}\,\mathrm{SE}(\hat{\beta}_j), \qquad \text{odds ratio } e^{\beta_j} : \quad \left[e^{\text{lower}},\, e^{\text{upper}}\right]$$

- Why z and not t? Maximum likelihood estimators are approximately normal for large n – as for the known-variance case of Part 3.
- The CI for the odds ratio is obtained by **exponentiating the bounds** – it is not symmetric around $e^{\hat{\beta}_j}$.

Inference – balance and student status

Balance ($\hat{\beta}_1 = 0.005499$, $\mathrm{SE} = 0.000220$)

$$Z = \frac{0.005499}{0.000220} = 24.95, \qquad p < 2 \cdot 10^{-16}, \qquad \mathrm{CI}_{0.95}(\beta_1) = [\,0.005067\,;\,0.005931\,]$$

Per 100 \$: odds ratio $e^{0.5499} = 1.733$, CI $[e^{0.5067};\, e^{0.5931}] = [\,1.660\,;\,1.810\,]$ – every 100 \$ of balance raises the odds of default by 66 to 81 %.

Student ($\hat{\beta} = 0.4049$, $\mathrm{SE} = 0.1150$), `default` $\sim$ `student`

$$Z = \frac{0.4049}{0.1150} = 3.52, \qquad p = 0.0004, \qquad e^{0.4049} = 1.50, \quad \mathrm{CI} = [\,1.20\,;\,1.88\,]$$

Students have 50 % higher odds of default – *on their own* (more on this in Section 3).

Exercise – coefficients on the odds scale

Problem statement

The logistic regression of default on balance gives $\hat{\beta}_1 = 0.0055$ (per \$) with $\mathrm{SE}(\hat{\beta}_1) = 0.00022$.

1. By how much do the *log-odds* change for 500 \$ more balance?
2. By which factor do the *odds* change for 500 \$ more balance?
3. Test $H_0 : \beta_1 = 0$ at $\alpha = 0.05$.
4. Why can we state a constant effect on the odds but not on the probability?

Solution – coefficients on the odds scale

(1) $\Delta\eta = 0.0055 \cdot 500 = \mathbf{2.75}$. **(2)** $e^{2.75} = \mathbf{15.6}$: the odds are about 16 times as large.

(3) $Z = 0.0055/0.00022 = 25.0 > 1.96 \Rightarrow$ reject H_0: balance is a highly significant predictor.

(4) The log-odds are linear in x, so a fixed Δx always shifts them by $\beta_1 \Delta x$ and multiplies the odds by $e^{\beta_1 \Delta x}$. The probability is the S-shaped logistic function of the log-odds:

start	odds before	odds after ($\times 15.6$)	probability after
$p = 0.10$	0.111	1.74	0.63 (+53 points)
$p = 0.90$	9	141	0.993 (+9 points)

Overview

1 Classification Problems

2 Logistic Regression

3 Multiple Logistic Regression
- The model
- Confounding: the student paradox
- Exercises

4 From Probabilities to Decisions

5 Evaluating a Classifier

6 Summary

Several explanatory variables

Model

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Everything from Part 4 carries over:

- β_j is the change of the log-odds per unit of x_j **holding all other variables fixed**; e^{β_j} is the corresponding odds ratio.
- Qualitative variables enter as **dummies** (e.g. `student[T.Yes]` $= 1$ for students).
- Simple and multiple coefficients can differ completely – **confounding**.

Multiple logistic regression with Python

```
g3 = smf.logit("y ~ balance + income + student", data=df).fit(disp=0)
print(g3.summary().tables[1])
```

```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept    -10.8690      0.492    -22.079      0.000     -11.834      -9.904
student[T.Yes]  -0.6468      0.236     -2.738      0.006      -1.110      -0.184
balance       0.0057      0.000     24.737      0.000       0.005       0.006
income     3.033e-06   8.2e-06      0.370      0.712    -1.3e-05    1.91e-05
==============================================================================
```

- **Balance:** highly significant, positive.
- **Income:** not significant given balance and student ($p = 0.71$).
- **Student:** significant and now *negative*: $e^{-0.6468} = 0.524$.

The student paradox

Alone: `default` $\sim$ `student`

$\hat{\beta}_{\text{student}} = +0.405$ ($p = 0.0004$)
Students default *more often*:
4.31 % vs. 2.92 %.

Together with balance and income

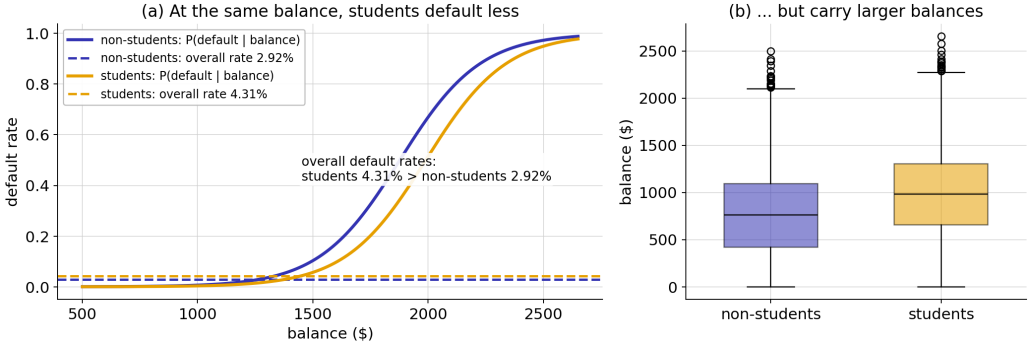
$\hat{\beta}_{\text{student}} = -0.647$ ($p = 0.006$)
At the *same* balance, students default *less often*.

How can both be true?

Students carry **larger balances** on average (988 \$ vs. 772 \$) – and a large balance is the main risk factor. Balance is a **confounder** of the student effect.

In words: The same phenomenon as the newspaper budget in Part 4: the simple coefficient answers a different question than the multiple one.

The student paradox visualised



- (a) For every balance, the student curve lies *below* the non-student curve; the overall rates (dashed) point the other way.
- (b) Students are concentrated at higher balances.

Which coefficient is "right"?

Both – they answer different questions

- **Simple model:** "Is a randomly chosen student riskier than a randomly chosen non-student?" – *yes* (relevant if nothing else is known about the applicant).
- **Multiple model:** "Is a student riskier than a non-student *with the same balance and income*?" – *no* (relevant for an individual credit decision, where the balance is known).

Key takeaway

Always ask *which variables are held fixed* before interpreting a coefficient – in linear and in logistic regression.

Exercise – inference in the multiple model

Problem statement

In the model `default` $\sim$ `balance + income + student`:

	$\hat{\beta}_j$	$\mathrm{SE}(\hat{\beta}_j)$
income (per \$)	$3.033 \cdot 10^{-6}$	$8.203 \cdot 10^{-6}$
`student[T.Yes]`	-0.6468	0.2363

1. Test $H_0 : \beta_{\text{income}} = 0$ at $\alpha = 0.05$.
2. Compute a 95 % CI for the odds ratio of 10 000 \$ more income.
3. Compute the odds ratio for students and its 95 % CI. Interpret it.

Solution – inference in the multiple model

(1) $Z = \dfrac{3.033}{8.203} = 0.37 < 1.96 \Rightarrow$ keep H_0 ($p = 0.71$): given balance and student status, income adds no detectable information.

(2) Per 10 000 \$: $\hat{\beta} = 0.0303$, $\mathrm{SE} = 0.0820$:

$$0.0303 \pm 1.96 \cdot 0.0820 = [-0.130\,;\, 0.191] \;\Rightarrow\; \mathrm{OR} \in [e^{-0.130};\, e^{0.191}] = [0.878\,;\, 1.211]$$

The interval contains 1 ("no change of the odds") – consistent with (1).

(3) $e^{-0.6468} = \mathbf{0.524}$; $-0.6468 \pm 1.96 \cdot 0.2363 = [-1.110\,;\, -0.184] \Rightarrow \mathrm{OR} \in [0.330\,;\, 0.832]$.

Interpretation

At the same balance and income, a student's odds of default are about half (48 % lower) those of a non-student; the CI excludes 1.

Exercise – logistic regression from A to Z

Problem statement

Use the multiple model with (rounded) coefficients

$$\hat{\eta} = -10.869 + 0.00574 \cdot \text{balance} + 0.003 \cdot \text{income} - 0.6468 \cdot \text{student}$$

(income in 1000 \$, student $= 1$ for students). Two customers:

- **A**: non-student, balance 1000 \$, income 40 000 \$;
- **B**: student, balance 2000 \$, income 30 000 \$.

1. Compute $\hat{\eta}$ and $\hat{p}$ for A and B.
2. By which factor do the odds change for 100 \$ more balance?
3. At which balance does a non-student with 40 000 \$ income reach $\hat{p} = 0.5$? And a student with the same income?

Solution (1/2) – logistic regression from A to Z

(1) Log-odds and probabilities (watch the units: income in 1000 $)

$$\hat{\eta}_A = -10.869 + 0.00574 \cdot 1000 + 0.003 \cdot 40 = -10.869 + 5.740 + 0.120 = -5.009$$
$$\hat{\eta}_B = -10.869 + 0.00574 \cdot 2000 + 0.003 \cdot 30 - 0.6468 = 0.054$$

$$\hat{p}_A = \frac{e^{-5.009}}{1 + e^{-5.009}} = \mathbf{0.0066}, \qquad \hat{p}_B = \frac{e^{0.054}}{1 + e^{0.054}} = \mathbf{0.514}$$

A is a very safe customer, B is right at the knife edge. (Python with unrounded coefficients, `g3.predict(...)`: 0.0066 and 0.512.)

(2) $e^{0.00574 \cdot 100} = e^{0.574} = \mathbf{1.78}$: 100 $ more balance raises the odds by 78 % – whatever the starting point.

Solution (2/2) – logistic regression from A to Z

(3) Decision boundary: $\hat{p} = 0.5 \Leftrightarrow \hat{\eta} = 0$

Non-student, income 40:

$$0 = -10.869 + 0.00574 \cdot \text{balance} + 0.120 \;\Rightarrow\; \text{balance} = \frac{10.869 - 0.120}{0.00574} \approx \mathbf{1873}\,\$$$

Student, income 40:

$$\text{balance} = \frac{10.869 - 0.120 + 0.6468}{0.00574} \approx \mathbf{1985}\,\$$$

In words: A student needs about 112 \$ more balance to reach the same 50 % risk – the negative student coefficient, expressed in dollars.

Common mistake

Inserting the income in dollars: $0.003 \cdot 40\,000 = 120$ would push every probability to 1.

Overview

1 Classification Problems
2 Logistic Regression
3 Multiple Logistic Regression
4 From Probabilities to Decisions
5 Evaluating a Classifier
6 Summary

The classification rule

Rule with threshold t

$$\hat{y} = \begin{cases} 1 \text{ (“default”)} & \text{if } \hat{p}(x) > t, \\ 0 \text{ (“no default”)} & \text{otherwise.} \end{cases}$$

The usual default is $t = 0.5$: assign the more probable class.

Threshold on the log-odds scale

Because the logistic function is increasing,

$$\hat{p}(x) > t \quad \Longleftrightarrow \quad \hat{\eta}(x) > \log \frac{t}{1-t}$$

$t = 0.5$ corresponds to $\hat{\eta} > 0$; $t = 0.2$ to $\hat{\eta} > \log(0.25) = -1.386$.

Decision boundary – default and balance

Model: $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$.

Threshold	log-odds cut-off	balance cut-off
$t = 0.5$	$\hat{\eta} > 0$	$10.65/0.0055 = 1936\,\$$
$t = 0.2$	$\hat{\eta} > \log(0.2/0.8) = -1.386$	$(10.65 - 1.386)/0.0055 = 1684\,\$$

In words: Lowering the threshold moves the boundary to the left: more defaulters are caught – but more good customers are rejected as well.

Key takeaway

The threshold is not a statistical but a **business decision**: it depends on the cost of a missed defaulter compared with the cost of a rejected good customer.

Exercise – from probability to decision

Problem statement

Use $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$. Three applicants have balances of 1800, 2000 and 2200 \$.

1. Compute the predicted probability of default for each.
2. Classify them with the threshold $t = 0.5$.
3. The bank lowers the threshold to $t = 0.2$. Which classifications change?

Solution – from probability to decision

(1) Probabilities

$$
\begin{aligned}
1800: \quad & \hat{\eta} = -10.65 + 9.90 = -0.75, && \hat{p} = \frac{e^{-0.75}}{1+e^{-0.75}} = 0.321 \\
2000: \quad & \hat{\eta} = -10.65 + 11.00 = \ \ 0.35, && \hat{p} = \frac{e^{0.35}}{1+e^{0.35}} = 0.587 \\
2200: \quad & \hat{\eta} = -10.65 + 12.10 = \ \ 1.45, && \hat{p} = \frac{e^{1.45}}{1+e^{1.45}} = 0.810
\end{aligned}
$$

(2) $t = 0.5$ (boundary 1936 \$): 1800 $\rightarrow$ **No**, 2000 $\rightarrow$ **Yes**, 2200 $\rightarrow$ **Yes**.

(3) $t = 0.2$ (boundary 1684 \$): the applicant with 1800 \$ ($\hat{p} = 0.321 > 0.2$) flips to **Yes**; the other two stay **Yes**.

In words: Equally spaced balances (+200 \$) give unequal probability steps (+0.27, +0.22) – the S-curve is non-linear.

Overview

1 Classification Problems

2 Logistic Regression

3 Multiple Logistic Regression

4 From Probabilities to Decisions

5 Evaluating a Classifier
- Confusion matrix
- ROC curve and AUC
- Training and test data

6 Summary

Why accuracy alone is not enough

The trivial classifier

Only 3.33 % of the customers default. The "classifier" that **always predicts "no default"** is right for 96.67 % of the customers – and detects not a single defaulter.

Imbalanced classes

With a rare class, a high accuracy is easy to achieve and says little. We must look at the two kinds of errors **separately** – they also have very different costs.

In words: As in Part 3 there are two ways to be wrong: a false alarm (Type I) and a miss (Type II).

The confusion matrix

Actual \ Predicted	No	Yes
No	**TN** true negative	**FP** false positive
Yes	**FN** false negative	**TP** true positive

Measures

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{n}$$

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Sensitivity** (recall): share of actual defaulters that are caught. **Specificity**: share of non-defaulters correctly left alone.
- **Precision**: share of flagged customers that really default. Error rate $= 1 -$ accuracy.

Confusion matrix – Default data, $t = 0.5$

Model: `default` $\sim$ `balance + income + student`, $t = 0.5$, all $n = 10\,000$ customers:

	Pred. No	Pred. Yes	Total
Actual No	9627	40	9667
Actual Yes	228	105	333
Total	9855	145	10 000

Metrics

Accuracy $= \frac{9627+105}{10\,000} = 0.973$ Sensitivity $= \frac{105}{333} = 0.315$

Specificity $= \frac{9627}{9667} = 0.996$ Precision $= \frac{105}{145} = 0.724$

In words: An accuracy of 97.3 % – but 228 of 333 defaulters (68 %) are missed.

Exercise – reading a confusion matrix

Problem statement

The bank lowers the threshold of the same model to $t = 0.2$:

	Pred. No	Pred. Yes	Total
Actual No	9390	277	9667
Actual Yes	130	203	333

1. Compute accuracy, error rate, sensitivity, specificity and precision.
2. Compare with $t = 0.5$ (previous slide). Which threshold would you recommend to the bank?

Solution – reading a confusion matrix

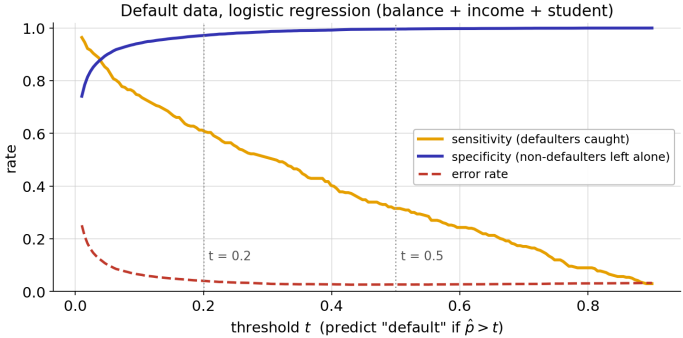
(1) Metrics at $t = 0.2$ (with $t = 0.5$ for comparison):

	$t = 0.5$	$t = 0.2$	
Accuracy	0.973	0.959	$= (9390 + 203)/10\,000$
Error rate	0.027	0.041	$= 1 - 0.959$
Sensitivity	0.315	0.610	$= 203/333$
Specificity	0.996	0.971	$= 9390/9667$
Precision	0.724	0.423	$= 203/(203 + 277)$

(2) With $t = 0.2$ the bank catches almost twice as many defaulters (203 instead of 105) at the price of 237 additional false alarms. If a missed default costs much more than a rejected good customer, $t = 0.2$ is the better choice – although its accuracy is lower.

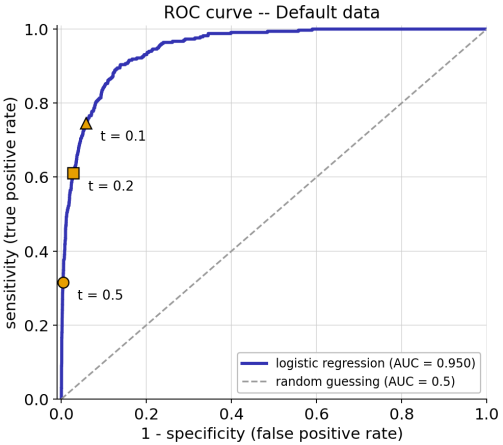
In words: Do not confuse sensitivity and precision: both have TP on top, but divide by the *actual* (333) or the *predicted* (480) positives.

The threshold trade-off



- Lower $t \Rightarrow$ sensitivity $\uparrow$, specificity $\downarrow$ – and vice versa.
- The error rate is minimal around $t = 0.5$, but that is not necessarily the *cheapest* threshold.

The ROC curve



Construction

For *every* threshold t, plot the pair

$$\big(1 - \text{specificity},\ \text{sensitivity}\big).$$

- Each point is one threshold; the curve shows all of them.
- The diagonal is random guessing.
- The closer the curve gets to the **top-left corner**, the better the classifier.

AUC – area under the ROC curve

Definition and interpretation

The AUC is the area under the ROC curve. Equivalently,

$$\text{AUC} = P\big(\hat{p}(\text{random defaulter}) > \hat{p}(\text{random non-defaulter})\big).$$

- AUC $= 0.5$: no better than random; AUC $= 1$: perfect ranking.
- **Default data:** AUC $= 0.950$ – a randomly chosen defaulter receives a higher predicted probability than a randomly chosen non-defaulter in 95 % of all pairs.
- The AUC summarises **all thresholds** at once – changing t moves along the curve but never changes the AUC.

Key takeaway

The ROC curve and AUC evaluate the *model*; the confusion matrix evaluates one *decision rule* (one threshold).

Exercise – threshold, ROC and AUC

Problem statement

For the Default model: $t = 0.5$ gives sensitivity 0.315 and specificity 0.996; $t = 0.2$ gives sensitivity 0.610 and specificity 0.971.

1. Locate both operating points on the ROC axes.
2. How does the point move when the threshold drops from 0.5 to 0.2?
3. A colleague hopes to improve the AUC by choosing a better threshold. Comment.

Solution – threshold, ROC and AUC

(1) $t = 0.5$: $(1 - 0.996,\ 0.315) = (0.004,\ 0.315)$;
$t = 0.2$: $(1 - 0.971,\ 0.610) = (0.029,\ 0.610)$.

(2) The point moves **up and to the right along the same curve**: a small loss in specificity ($+0.025$ on the x-axis) buys a large gain in sensitivity ($+0.295$).

(3) Impossible: the threshold only selects a point *on* the curve. The AUC belongs to the model's ranking of the customers – only a better *model* (better variables) can raise it.

Practice

Choose the model by its AUC, then choose the threshold by the costs of the two error types.

Honest evaluation: training and test data

Problem

So far we evaluated the model on the *same* data used to fit it. This is **optimistic**: the model has adapted to exactly these customers. What counts is the performance on **new** customers.

Solution: split the data

- **Training data** (e.g. 70 %): estimate the model.
- **Test data** (the remaining 30 %): compute the confusion matrix and the AUC – data the model has never seen.

Default data: 7000 training, 3000 test customers (`random_state=1`)

At $t = 0.5$ the model catches 34 of 112 test defaulters, at $t = 0.2$ already 67. Test AUC $= 0.956$ (in-sample on all data: 0.950) – with only four coefficients there is little to overfit, and chance can even push a test value above the in-sample one.

Exercise – training vs. test data

Problem statement

The model `default ~ balance + income + student` is fitted on 7000 training customers and applied to the 3000 test customers (112 of them defaulted):

	TN	FP	FN	TP
Test data, $t = 0.5$	2878	10	78	34
Test data, $t = 0.2$	2824	64	45	67

1. Compute accuracy, sensitivity, specificity and precision for both thresholds.
2. On all 10 000 customers (training = evaluation data) the sensitivity at $t = 0.5$ was 0.315. Compare and explain.

Solution – training vs. test data

(1) Test-data metrics:

	$t = 0.5$	$t = 0.2$
Accuracy	$\frac{2878+34}{3000} = 0.971$	$\frac{2824+67}{3000} = 0.964$
Sensitivity	$\frac{34}{112} = 0.304$	$\frac{67}{112} = 0.598$
Specificity	$\frac{2878}{2888} = 0.997$	$\frac{2824}{2888} = 0.978$
Precision	$\frac{34}{44} = 0.773$	$\frac{67}{131} = 0.511$

(2) On new customers the model catches 30.4 % of the defaulters at $t = 0.5$, slightly less than the 31.5 % on the data it was fitted on. In-sample figures tend to be **optimistic**; here the gap is small (a simple model, 7000 training customers), and with only 112 test defaulters part of any difference is chance.

In words: Report the test-data metrics – they describe what the bank can expect for future applicants.

Classification with Python

Predicted probabilities and confusion matrix

```
phat = g3.predict(df)              # P(default = Yes); log-odds: which="linear"
print(pd.crosstab(df.default, phat > 0.5))     # t = 0.5: 9627 40 / 228 105
```

Training and test data, test AUC

```
from sklearn.metrics import roc_auc_score
train = df.sample(n=7000, random_state=1)      # 70 % training data
test = df.drop(train.index)                    # remaining 30 % test data
g_tr = smf.logit("y ~ balance + income + student", data=train).fit(disp=0)
p_te = g_tr.predict(test)                      # test-set probabilities
print(pd.crosstab(test.default, p_te > 0.5))   # 2878 10 / 78 34
print(roc_auc_score(test.y, p_te))             # test AUC 0.956
```

Overview

1 Classification Problems
2 Logistic Regression
3 Multiple Logistic Regression
4 From Probabilities to Decisions
5 Evaluating a Classifier
6 Summary

Summary 1 – logistic regression

Model

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p, \qquad p(x) = \frac{e^\eta}{1+e^\eta}$$

estimated by **maximum likelihood**.

Interpretation of β_j (holding the other variables fixed)

- sign: direction of the effect on the probability,
- β_j: change of the log-odds per unit of x_j,
- e^{β_j}: **odds ratio** – factor by which the odds change per unit of x_j.

Inference

$Z = \hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j) \approx N(0,1)$, CI $\hat{\beta}_j \pm z_{1-\alpha/2}\mathrm{SE}(\hat{\beta}_j)$, CI for the odds ratio: exponentiate the bounds.

Summary 2 – decisions and evaluation

Classification rule

$\hat{y} = 1$ if $\hat{p}(x) > t \;\Leftrightarrow\; \hat{\eta}(x) > \log \frac{t}{1-t}$; the threshold t is a business decision.

Measure	Formula	Question
Accuracy	$(\mathrm{TP} + \mathrm{TN})/n$	share of correct decisions
Sensitivity	$\mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$	share of positives caught
Specificity	$\mathrm{TN}/(\mathrm{TN} + \mathrm{FP})$	share of negatives left alone
Precision	$\mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$	share of alarms that are real
AUC	area under ROC curve	ranking quality over all thresholds

In words: Evaluate on **test data** the model has not seen.

Summary 3 – important Python commands

Logistic regression

```
g = smf.logit("y ~ x1 + x2",
              data=df).fit()
print(g.summary())     # z-tests
np.exp(g.params)       # odds ratios
np.exp(g.conf_int())   # CIs for ORs
g.predict(new)         # probabilities
```

Decisions and evaluation

```
phat = g.predict(df)
pd.crosstab(df.y, phat > 0.5)
roc_auc_score(df.y, phat)  # AUC
train = df.sample(n=7000,
                  random_state=1)
test = df.drop(train.index)
```

→ *Appendix: Bayes classifiers in sklearn:* `LinearDiscriminantAnalysis`, `GaussianNB`

Five classic pitfalls – final checklist

Forbidden sentence ×	**Correct version** ✓
"The odds ratio is 1.78, so the probability rises by 78 %."	The *odds* rise by 78 %; the change in probability depends on the starting point.
"Accuracy 97 % – an excellent classifier."	With 3 % defaulters, always saying "no" gives 97 %; check sensitivity and specificity.
"$t = 0.5$ is the correct threshold."	The threshold depends on the costs of the two errors.
"A better threshold improves the AUC."	The threshold moves along the ROC curve; only a better model raises the AUC.
"Students are riskier, so reject students."	At the same balance, students are *less* risky – ask what is held fixed.

Looking back: the lecture in one slide

Part	Key idea
1 – Descriptive Statistics	Summarise data by location and dispersion ($\bar{x}$, $\tilde{x}$, s, v).
2 – Graphical Representations	Show the distribution: empirical CDF, histogram, boxplot.
3 – Statistical Inference	Infer from sample to population: estimate $\pm$ critical value $\times$ SE; tests and p-values.
4 – Regression	Explain a metric Y by several X – with t-tests, CIs and diagnostics.
5 – Classification	Explain a categorical Y: logistic regression, decision rules, evaluation.

Key takeaway

One thread through all five parts: every number computed from data is an **estimate** – describe it, picture it, and report its uncertainty.

Overview

7 Appendix: Bayes' Theorem and Generative Classifiers
- A.1 Bayes' theorem for classification
- A.2 Naive Bayes
- A.3 Linear discriminant analysis
- A.4 Comparison

Appendix – what is in here, and why

Optional material

The main part covers the standard classification workflow with logistic regression. The appendix shows a second family of classifiers that start from **Bayes' theorem** – useful for small samples, many features or more than two classes.

Topic	**Content**
A.1 Bayes' theorem	Prior $\times$ likelihood $\rightarrow$ posterior; the Bayes classifier
A.2 Naive Bayes	Independent features: the spam filter (exercise)
A.3 LDA	Normal classes with common variance (exercise)
A.4 Comparison	Logistic regression, LDA and naive Bayes; Python commands

Two ways to obtain $P(Y = k \mid x)$

Discriminative

Model $P(Y = 1 \mid x)$ **directly**.

Example: logistic regression.

Generative

Model how x is distributed **within each class** and combine it with the class frequencies via **Bayes' theorem**.

Examples: naive Bayes, LDA.

When generative models are useful

Small samples, well separated classes, more than two classes – and they make the **base rate** (prior) explicit, which matters whenever one class is rare.

Bayes' theorem in the classification setting

Posterior probability of class k

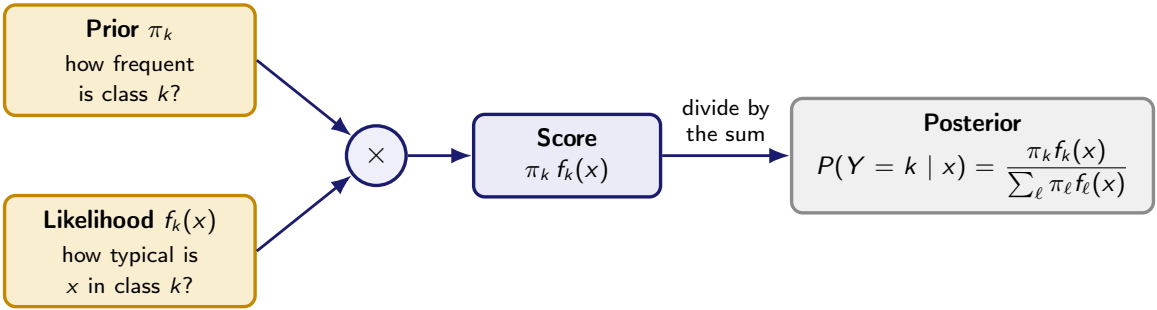
$$P(Y = k \mid X = x) = \frac{\pi_k \, f_k(x)}{\sum_\ell \pi_\ell \, f_\ell(x)}$$

Symbol	Meaning
$\pi_k = P(Y = k)$	**prior**: share of class k (e.g. 3.33 % defaulters)
$f_k(x)$	**likelihood**: how typical is x for class k?
$P(Y = k \mid x)$	**posterior**: probability of class k given x

Bayes classifier

Assign x to the class with the **largest posterior** – for two classes: $\hat{y} = 1$ if $P(Y = 1 \mid x) > 0.5$.

Bayes' theorem as a flow



Key takeaway

Every generative classifier is this flow – naive Bayes and LDA differ only in **how they model the likelihood** $f_k(x)$.

Naive Bayes: an extreme simplification

Assumption

Within each class, the explanatory variables are **independent**:

$$f_k(x) = f_{k1}(x_1) \cdot f_{k2}(x_2) \cdots f_{kp}(x_p).$$

- Instead of one p-dimensional distribution, only p one-dimensional ones have to be estimated – e.g. simple proportions for yes/no features.
- The assumption is almost always *wrong* – yet the *ranking* of the cases is often good enough to classify well.

In words: Classic application: spam filters – each word contributes its own factor to the score.

Exercise – naive Bayes spam filter

Problem statement

A spam filter has priors $P(\text{Spam}) = 0.4$, $P(\text{Ham}) = 0.6$ and two binary features: F_1 ("contains the word *free*") and F_2 ("contains a link").

	$P(F_1 = 1 \mid \cdot)$	$P(F_2 = 1 \mid \cdot)$
Spam	0.6	0.7
Ham	0.1	0.2

A new e-mail contains *both* features.

1. Compute the score $\pi_k \, f_k(x)$ of each class.
2. Compute $P(\text{Spam} \mid F_1 = 1, F_2 = 1)$ and classify the e-mail.

Solution – naive Bayes spam filter

(1) Scores (prior × product of the feature probabilities)

$$\text{Spam: } 0.4 \cdot 0.6 \cdot 0.7 = 0.168, \qquad \text{Ham: } 0.6 \cdot 0.1 \cdot 0.2 = 0.012$$

(2) Posterior

$$P(\text{Spam} \mid F_1, F_2) = \frac{0.168}{0.168 + 0.012} = \frac{0.168}{0.180} = \mathbf{0.933} > 0.5 \Rightarrow \textbf{Spam}$$

In words: The two features together are 21 times as likely under Spam as under Ham (0.42/0.02); this overwhelms the prior, which favoured Ham.

Common mistake

Ignoring the prior: $0.42/(0.42 + 0.02) = 0.955$ overstates the spam probability. With rare positives the prior can even flip the decision.

Linear discriminant analysis (LDA) with one variable

Assumption

Within each class, X is **normally distributed** with mean μ_k and a **common** variance σ^2: $X \mid Y = k \sim N(\mu_k, \sigma^2)$.

Discriminant score and decision boundary (two classes)

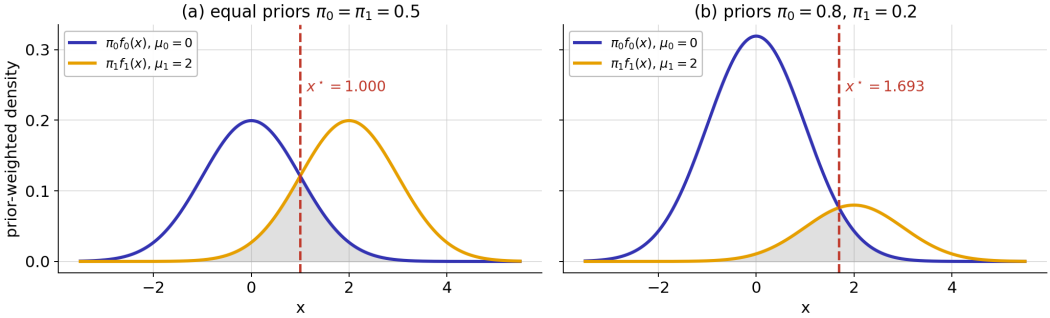
Inserting the normal densities into Bayes' theorem and taking logs gives a score **linear** in x:

$$\delta_k(x) = x\,\frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log \pi_k, \qquad x^\star = \frac{\mu_0 + \mu_1}{2} - \frac{\sigma^2}{\mu_1 - \mu_0}\,\log\frac{\pi_1}{\pi_0}.$$

Assign the class with the larger score; at $x^\star$ both scores are equal.

In words: Equal priors: the boundary is the midpoint of the means. A rare class 1 pushes the boundary towards μ_1.

LDA with one variable – in pictures



- LDA compares the **prior-weighted densities** $\pi_k f_k(x)$; the boundary lies where they cross.
- The grey overlap is the error that *no* classifier can avoid.

Exercise – LDA in one dimension

Problem statement

Two classes with $X \mid Y = k \sim N(\mu_k, \sigma^2)$, $\sigma^2 = 1$, $\mu_0 = 0$, $\mu_1 = 2$.

1. With equal priors, write down $\delta_0(x)$ and $\delta_1(x)$ and find the decision boundary.
2. Classify $x = 1.5$.
3. Now take $\pi_0 = 0.8$, $\pi_1 = 0.2$. Recompute the boundary and reclassify $x = 1.5$.

Solution – LDA in one dimension

(1)–(2) Equal priors

$\delta_0(x) = 0 + \log 0.5$, $\delta_1(x) = 2x - 2 + \log 0.5$. The equal $\log \pi_k$ terms cancel: $0 = 2x - 2 \Rightarrow x^\star = 1$ (the midpoint of the means). $x = 1.5 > 1 \Rightarrow$ **class 1**.

(3) Unequal priors

$\delta_0 = \log 0.8 = -0.223$, $\delta_1 = 2x - 2 + \log 0.2 = 2x - 3.609$:

$$-0.223 = 2x - 3.609 \;\Rightarrow\; x^\star = 1.693 \qquad \left(\text{check: } 1 - \tfrac{1}{2}\log\tfrac{0.2}{0.8} = 1 + 0.693 \;\checkmark\right)$$

Now $x = 1.5 < 1.693 \Rightarrow$ **class 0**.

In words: Same observation, opposite decision – purely because class 1 became rare: the rarer class needs stronger evidence.

Logistic regression, LDA and naive Bayes

	Logistic regression	**LDA**	**Naive Bayes**
Models	$P(Y = 1 \mid x)$ directly	normal f_k, common variance	independent features
Boundary	linear in x	linear in x	flexible
Strength	few assumptions; interpretable odds ratios, tests	stable for small n, several classes	very many features, small n
Weakness	unstable if classes are perfectly separated	wrong if normality fails	miscalibrated probabilities

Default data (all customers, $t = 0.5$)

LDA (`balance + student`) catches 81 of 333 defaulters with 23 false alarms; logistic regression catches 105 with 40 false alarms – similar trade-offs, AUC around 0.95 for both.

Bayes classifiers with Python

Linear discriminant analysis

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = pd.get_dummies(df[["balance", "student"]], drop_first=True)
lda = LinearDiscriminantAnalysis().fit(X, df.default)
print(pd.crosstab(df.default, lda.predict(X)))    # 9644 23 / 252 81
```

Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
nb = GaussianNB().fit(X, df.default)        # normal f_kj for every feature
print(pd.crosstab(df.default, nb.predict(X)))
```

Appendix summary – Bayes classifiers

Bayes' theorem

$$P(Y = k \mid x) = \frac{\pi_k \, f_k(x)}{\sum_\ell \pi_\ell \, f_\ell(x)} \qquad \text{(posterior} \propto \text{prior} \times \text{likelihood)}$$

- **Naive Bayes:** $f_k(x) = \prod_j f_{kj}(x_j)$ – independent features within each class.
- **LDA:** normal f_k with common variance; one variable: boundary $x^\star = \frac{\mu_0+\mu_1}{2} - \frac{\sigma^2}{\mu_1-\mu_0} \log \frac{\pi_1}{\pi_0}$.
- A rare class needs stronger evidence – the prior shifts the boundary.